# AI Models — Integration & Expected Behavior (README_AI)

This document defines exactly how each AI model is used in the AI Forensic Duplicate Detection CLI (Dupli-HQ), what to expect from its outputs, and where it plugs into the system. Use this as the source of truth when extending or troubleshooting the AI layer.

---

## 1) Big-Picture Integration Map

**Primary entry points**: - `commands.py`: one-shot CLI for `compare`, `snapshot`, `duplicates`, `tracker`. - `cli_shell.py`: interactive shell (`dupli-hq>`) with boot diagnostics.

**Automation/analysis**: - `daily_snapshot.py`: produces snapshot lines with `AIHASH` (SHA-256 + optional embedding) and compares with last snapshot. - `scan_duplicates.py`: finds exact and near duplicates using cryptographic hashes, pHash, and embeddings.

**Model loading** is centralized via `loader.py`, which dynamically imports modules from `src/ai_model/` and exposes: - `<model>_model.load_model()` → returns the ready-to-use model (and tokenizer/ preprocess if required) - `<model>_model.get_transform()` → returns input transforms, if applicable - `<model>_model.extract_features(...)` / `extract_features_from_file(...)` → returns a NumPy vector (embedding)

---

## 2) File-Type Routing & Decision Logic

**Image files** - **Snapshot**: By default uses **CLIP** to produce an image embedding (in addition to the mandatory SHA-256). - **Duplicate scan**: pHash pre-filter (Hamming distance) → then **DINOv2** and **ResNet50** embeddings; the maximum cosine similarity across these vectors is used to confirm near duplicates. - **Compare mode**: When `--auto`, `commands.py` selects **CLIP** for image–image comparisons.

**Text files** (plain text, markdown, logs) - **Snapshot**: Uses **SBERT-Deep** to generate a semantic embedding next to SHA-256. - **Duplicate scan**: Uses **SBERT-Deep and CodeBERT**; vectors are concatenated before cosine similarity to reduce false positives. - **Compare mode**: With `--auto`, selects **SBERT-Deep**.

**Code files** (`.py`, `.c`, `.cpp`, `.java`, `.js`, `.sh`) - **Snapshot**: "Hybrid"—computes **SBERT-Deep** and **CodeBERT** and concatenates when both succeed. - **Duplicate scan**: Same hybrid approach; concatenated vector is compared with cosine similarity. - **Compare mode**: With `--auto`, selects **CodeBERT`**.

**Binary/archives** - **Snapshot**: SHA-256 only. - **Duplicate scan**: SHA-256 equality only (no embeddings).

**Thresholds** - **Duplicate scan (AI)**: default cosine threshold `0.75` for near-duplicate acceptance (image/text/code). - **pHash screen**: 64-bit pHash, reject if Hamming distance exceeds `PHASH_MAX_DISTANCE` derived from similarity threshold `0.40`. - **Snapshot diff (embeddings)**: only used when byte hashes match; flag if cosine similarity `< 0.999999` (indicates semantic drift despite identical bytes—rare; typically informational).

---

# 3) Model-by-Model Specs

### A) CLIP (ViT-B/32, LAION)

- **Purpose**: Image embeddings for cross-modal robustness and quick perceptual checks.
- **I/O**: Returns a 512-d float vector (NumPy).
- **Transforms**: Provided by `open_clip` (`create_model_and_transforms`).
- **Used in**: Snapshot (image), Compare `--auto` (image).
- **Notes**: Model is run in eval mode; ensure RGB conversion; images are preprocessed to CLIP's expected size.

**Failure modes** - Cannot open image → embedding is skipped (snapshot still writes SHA-256). - Non-image file passed to CLIP → route via file-type detection, not CLIP.

### B) DINOv2 (ViT-Base, patch14, reg4)

- **Purpose**: High-fidelity image perceptual features, resilient to minor transforms.
- **I/O**: Returns a ~768-d float vector (NumPy) with classifier removed.
- **Transforms**: Resize to 518×518, normalized.
- **Used in**: Duplicate scan (image) after pHash screen; combined with ResNet50 and the best similarity is used.

**Failure modes** - Mismatched vector shapes (corrupt input) → pair is skipped.

### C) ResNet-50 (ImageNet features)

- **Purpose**: Complementary image features; stable baseline.
- **I/O**: 2048-d vector from penultimate layer.
- **Transforms**: ImageNet standard (224×224, mean/std normalize).
- **Used in**: Duplicate scan (image), combined with DINOv2.

**Neighbors**: ResNet-18 (512-d) and ResNet-101 (2048-d) are available for diagnostics or fallback.

### D) EfficientNet (B1, B3)

- **Purpose**: Optional/lightweight image feature extractors.
- **I/O**: B1 returns a 1280-d vector; B3 produces a deeper vector (flattened output).
- **Transforms**: Provided by torchvision weights.
- **Used in**: Available for experiments; not in default duplicate pipeline.

### E) SBERT (all-MiniLM-L6-v2)

- **Purpose**: Lightweight semantic text embeddings.
- **I/O**: Returns a fixed-size vector (MiniLM dimensionality) for entire file contents.
- **Used in**: Compare (when explicitly selected); legacy/lightweight scenarios.

### F) SBERT-Deep (all-mpnet-base-v2)

- **Purpose**: High-accuracy semantic text embeddings.
- **I/O**: Returns a 768-d vector (MPNet).
- **Used in**: Snapshot (text), Duplicate scan (text/code), Compare `--auto` (text).
- **Notes**: For code files, concatenated with CodeBERT to capture structure + semantics.

### G) CodeBERT (microsoft/codebert-base)

- **Purpose**: Code understanding; captures syntax-aware semantics.
- **I/O**: 768-d `[CLS]` vector (truncated to 512 tokens).
- **Used in**: Snapshot (code hybrid), Duplicate scan (text/code hybrid), Compare `--auto` (code).

**Hybrid concatenation (text/code)** - If both SBERT-Deep and CodeBERT vectors exist: concatenate into one long vector and compute cosine similarity on the concatenated representation.

---

## 4) Similarity Pipelines

### Images

1) **Exact match**: SHA-256 equality → `EXACT_DUPLICATE`. 2) **pHash screen**: Quick reject if Hamming distance too large. 3) **Deep check**: Compute DINOv2 and ResNet50 embeddings; take the **max** cosine similarity; if ≥ threshold → `NEAR_DUPLICATE (sim=…)`.

### Text / Code

1) **Exact match**: SHA-256 equality → `EXACT_DUPLICATE`. 2) **Embedding(s)**: SBERT-Deep and CodeBERT; concatenate vectors when both exist. 3) **Decision**: Cosine similarity ≥ threshold → `NEAR_DUPLICATE (sim=…)`.

**Edge guards** - Skip tiny/empty files; skip non-UTF-8 with decoding errors; skip zero-variance vectors; skip mismatched shapes.

---

## 5) Snapshot Semantics (AIHASH)

Each snapshot line stores **SHA-256** plus an optional embedding vector when the file type is supported. - Byte-level hash is **authoritative** for change detection. - Embeddings are used only as enrichment when bytes are equal (semantic drift detection) or to support downstream analytics. - Snapshots and diffs are stored under `reports/snapshots/` and `reports/diffs/`.

## 6) Tracker Semantics

The live tracker builds/loads a baseline snapshot and then continuously: - Generates a fresh snapshot on each interval - Computes differences (NEW / DELETED / MODIFIED) - Optionally runs a duplicate scan after changes - Logs alerts to `reports/tracker_alerts.txt`

---

## 7) Adding a New Model — Contract

Create `src/ai_model/your_model.py` with:

```
# Required
def load_model():
    ...  # return model or (tokenizer, model) and set eval() as needed

# Optional if images/text require preprocessing
def get_transform():
    ...  # return torchvision or custom transforms

# Required: one of these signatures
# For image models
def extract_features(path, model, transform):
    ...  # return 1D numpy vector

# For text/code models
def extract_features_from_file(path, model_or_tokenizer, maybe_model=None):
    ...  # return 1D numpy vector
```

Then register it in `loader.module_tree` and wire it in either `daily_snapshot.py` (for persistent AIHASH) and/or `scan_duplicates.py` (for duplicate detection). Keep output as a **1D NumPy array**.

---

## 8) Operational Expectations & Quality Notes

- All models run in **inference mode** ( `eval()` ), with deterministic outputs for the same inputs and versions.
- Cosine similarity is used across all embedding comparisons; keep vectors unnormalized if your extractor already returns normalized features; otherwise L2-normalize internally.
- For large folders, duplicate scan is **$O(N^2)$** per file-type group; the pHash screen for images is essential to limit deep comparisons.
- Reports are timestamped and never overwritten; downstream systems can parse JSON ( `reports/scan` ) or line-oriented diffs.

---

## 9) Minimal Usage Cheatsheet

- **Snapshot**: `python commands.py --mode snapshot --folder ./data`
- **Duplicates**: `python commands.py --mode duplicates --folder ./data`
- **Tracker**: `python commands.py --mode tracker --folder ./data`
- **Compare (auto)**: `python commands.py --mode compare --file1 a.png --file2 b.png --auto`

---

## 10) Dependencies (per model)

- **CLIP**: `open-clip-torch`, `Pillow`, `torch`.
- **DINOv2 / ResNets / EfficientNet**: `timm` (for DINOv2), `torchvision`, `Pillow`, `torch`.
- **SBERT / SBERT-Deep**: `sentence-transformers`.
- **CodeBERT**: `transformers`, `torch`.
- **Shared**: `numpy`, `scikit-learn` (cosine_similarity), `opencv-python` (for pHash), `tqdm`.

Lock versions for reproducibility.

---

**End of README_AI** — Keep this file versioned alongside code; when you change model versions or thresholds, update this document accordingly.